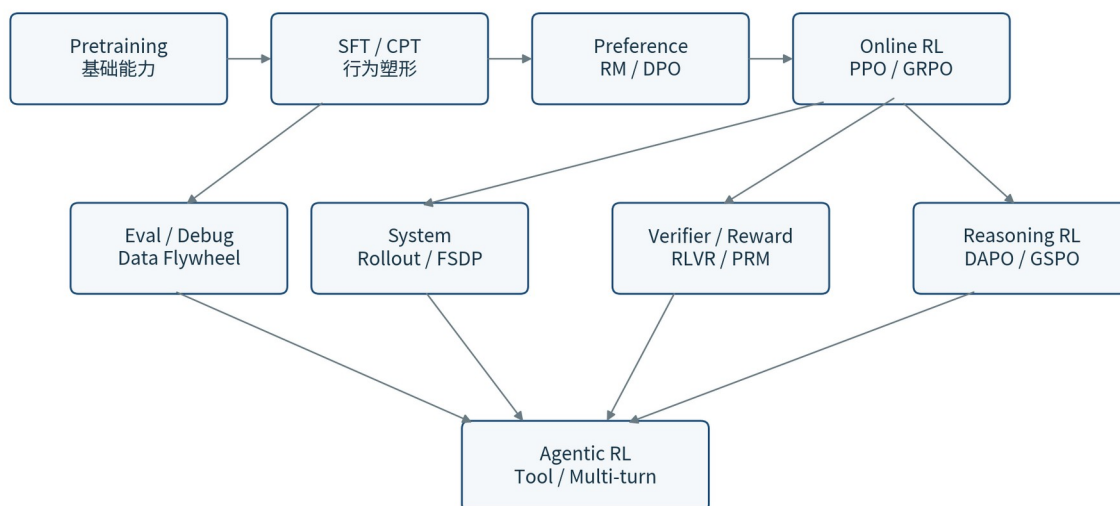


剑指 LLM 后训练 Offer

100 道真题级面试题 · 原理推导 · 工程实战 · 追问地图

面向：LLM 算法岗 / Post-Training / RLHF / Reasoning RL / Agentic RL

LLM 后训练能力地图：面试不是算法列表，而是一条闭环



核心闭环：数据 → 行为塑形 → 偏好/奖励 → 在线探索 → 系统扩展 → 评测与失败挖掘 → 再训练

本书定位 参考《剑指 Offer》的“题目—思路—解法—追问—举一反三”学习结构，但不复制其文字或题目。重点建立后训练算法的 know-why 与 know-how：为什么这样设计、哪里会坏、怎么调、系统如何跑。

版本：2026.09 · 基于公开论文、官方技术报告与公开面经整理

序言：不要背 100 答案，要建立 10 条推理链

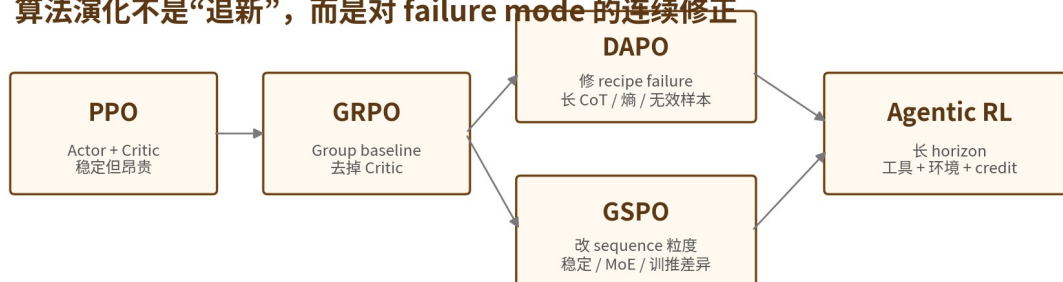
2026 年的大模型后训练面试已经明显从“RLHF / DPO 八股”进入“算法 + 数据 + reward + distributed system + Agent environment”的联合考察。公开面经里，PPO 的 clip 正负优势、GAE、GRPO 的 token credit、DAPO/GSPO、rollout 长尾、MoE routing mismatch、FSDP/ZeRO、TRL/verl 都已经出现。

因此本书不把题目当孤立知识点。每题按照“面试官为什么问 → 核心结论 → 深入理解 → 常见误区 → 追问”展开，并把题目放回算法演化链。你应该能够从问题约束自然推出算法，而不是从算法名反推故事。

公开面经属于候选人自述，不是企业官方题库。本书把可验证到的公开题目标为“公开真题”，其余标为“高频 / 推导 / 设计”，用来覆盖真实面试中的连续追问。

一条高分回答的骨架 Problem → Objective → Data → Reward → Exploration → Credit Assignment → Compute/System → Failure Mode → Algorithm Choice → Eval。

算法演化不是“追新”，而是对 failure mode 的连续修正



面试回答模板

任务与 reward 可验证性 → 数据是 offline 还是 online → 是否需要探索 → credit 粒度 → 计算/显存 → policy lag → 选择算法

高分答案：先讲约束，再讲选择；先讲 failure mode，再讲 trick。

如何使用本：四遍法

第一遍：30 秒结论：每题先只看“核心结论”，训练结构化口述。目标是任何题 30 秒内说清楚定义与 why。

第二遍：白板推导：对 PPO、GAE、DPO、GRPO、GSPO 相关题手写公式，并解释每个符号的工程含义。

第三遍：项目映射：把每题替换成你自己的模型规模、数据、reward、GPU、框架、指标与失败案例。

第四遍：连续追问：遮住答案，只看“面试官追问”。每题至少向下追三层，训练二面/三面的系统思维。

评级说明 L1 基础概念；L2 高频核心；L3 需要推理；L4 需要算法 + 工程；L5 研究/系统设计。

十章知识地图

章节	题号	主线	面试目标
1	1-10	SFT / 数据	能解释 post-training 目标与数据工程
2	11-20	Reward / RM	能设计 reward 并识别 hacking
3	21-30	PPO / GAE	能手推经典 RLHF
4	31-40	DPO family	能从 RLHF 推导 offline preference
5	41-50	GRPO	理解 group baseline 与 off-policy 工程
6	51-60	DAPO / GSPO	掌握 2026 前沿 failure mode
7	61-70	Reasoning / Verifier	理解 credit 与可验证性
8	71-80	RL System	能画数据流、估显存、解 rollout
9	81-90	Eval / Debug	能从曲线定位问题
10	91-100	Agentic RL	能做长程系统设计与项目答辩

面白板：必的核心公式

公式	表达式	一句话解释
SFT	$L_{\text{SFT}} = -\sum_t \log \pi_{\theta}(y_t x, y_{<t})$	teacher forcing / behavior cloning
RM	$L_{\text{RM}} = -\log \sigma(r_w - r_l)$	pairwise preference
PPO ratio	$r_t(\theta) = \pi_{\theta}(a_t s_t) / \pi_{\text{old}}(a_t s_t)$	distribution correction
PPO clip	$E[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)]$	pessimistic surrogate
GAE	$A_t = \sum_l (\gamma \lambda)^l \delta_{t+l}, \delta_t = r_t + \gamma V_{t+1} - V_t$	bias-variance trade-off
DPO	$-\log \sigma(\beta[(\log \pi_{\theta}(y_w) - \log \pi_{\text{ref}}(y_w)) - (\log \pi_{\theta}(y_l) - \log \pi_{\text{ref}}(y_l))])$	implicit reward
GRPO	$A_i = (r_i - \text{mean}(r_{\text{group}})) / \text{std}(r_{\text{group}})$	group-relative baseline
GSPO	$s_i = (\pi_{\theta}(y_i x) / \pi_{\text{old}}(y_i x))^{(1/ y_i)}$	sequence-level geometric mean ratio

白板技巧 写公式时同步解释“这个量从哪里来、解决什么偏差、假设什么时候失效”。公式本身只占 30%，其余 70% 是 know-why。

第一章 后训练全景与 SFT：先把“训练目标”想清楚

本章目标 完成面试题 1-10。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 01 什么是 Post-Training？为什么 Pretraining 后仍需要后训练？

[高频题] [L1]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：Pretraining 学“哪些 token 在上下文中可能出现”，后训练则把这种通用建模能力塑造成可控行为：指令遵循、偏好、安全、推理、工具使用与领域适应。

深入理解：把两种目标函数区分开：语言建模是分布拟合，post-training 是行为分布重塑。强 base model 的很多能力已经存在，后训练经常更像 capability elicitation 与 policy shaping，而非从零注入知识。

常见失分点：不要把 post-training 等同于 RLHF；SFT、偏好优化、RLVR、Agentic RL 都属于后训练。

面试官追问：如果 base model 已经很强，为什么少量高质量 SFT 仍然有效？

面试题 02 SFT 到底在学什么？为什么说它是 behavior cloning？

[高频题] [L1]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：SFT 用目标回答作为唯一轨迹做 teacher forcing，本质是最大化专家轨迹的条件似然。它最擅长输出格式、基本指令遵循、工具调用语法、风格与 cold-start reasoning。

深入理解：关键局限是它只“模仿看过的轨迹”，不会主动探索训练集没有的解法，也无法自然表达多个候选答案之间的相对偏好。

常见失分点：把 SFT 说成“让模型更聪明”过于粗糙；应区分能力、行为与分布。

面试官追问：为什么最佳 SFT checkpoint 不一定是最佳 RL initialization？

面试题 03 公开真题：SFT 数据如何筛选和采样？

[公开真题] [L2] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：按数据工程链路回答：来源治理 → 去重/泄漏 → 正确性 → 指令质量 → 难度 → 多样性 → 格式 → 安全 → 领域配比 → train/eval 隔离。

深入理解：面试官真正看的是你能否把“数据质量”变成可执行规则。对 reasoning 数据应额外检查最终答案、推理一致性、长度、语言混用和可验证性。采样可采用 temperature、domain cap、hard-example mining 与 capability-aware sampling。

常见失分点：只讲“清洗、去重、过滤低质”没有区分度；一定要讲筛选信号和错误样本如何回流。

面试官追问：如果只允许保留 20% SFT 数据，你会依据什么排序？ [I7]

面试题 04 SFT 数据是越多越好吗？如何理解 data quality × diversity × difficulty?

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：不是。大量低质量、重复或过易数据会放大错误梯度、模板化和 verbosity bias，并可能覆盖 base model 原有能力。

深入理解：有效数据价值更接近“边际训练信号”：正确、非重复、覆盖能力盲区、难度贴近当前 frontier 的样本贡献更高。对强模型而言，100 万条简单指令可能不如 5 万条高质量 hard cases。

常见失分点：只按 token 数汇报数据规模；忽略数据重复率、错误率和难度分布。

面试官追问：怎么做 data ablation 才能证明“数据质量”而不是训练步数带来的提升？

面试题 05 为什么 SFT 会造成 catastrophic forgetting 或 alignment tax?

[原理推导] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：因为优化分布从超大规模 pretraining distribution 收缩到较窄的 instruction distribution，梯度会持续推动参数适配新分布。

深入理解：缓解手段包括较小 LR、pretraining replay、领域混合、参数高效微调、early stopping 和回归评测。所谓 alignment tax 是行为对齐带来的通用能力/某些 benchmark 回退，应通过多目标数据与阶段化训练控制。

常见失分点：只会说“学习率太大”；forgetting 首先是分布与目标的问题。

面试官追问：为什么 replay 一定要有？什么情况下 LoRA 反而限制最终能力？

面试题 06 多领域 SFT 数据应该怎么配比？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：不应按原始数据量等比例采样，而应围绕业务价值、质量、稀缺性、难度、遗忘风险和当前能力短板设权重。

深入理解：可以将 mixture weight 视作控制训练梯度来源的策略变量；常见方法包括 temperature sampling、上限截断、curriculum、loss-aware sampling 与分阶段 mixture。

常见失分点：把“英语 50%、中文 50%”当成原则；真正应回答为什么这么配、如何在线调整。

面试官追问：某领域 loss 很高，是该加大权重还是数据质量有问题？如何区分？

面试题 07 为什么 instruction data 需要 prompt diversity，而不只是 response diversity?

[原理推导] [L1]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：同一意图的多种表达可以降低模板记忆，提高对意图而非表面形式的泛化。

深入理解：一个完善的数据扩增要同时改变措辞、上下文、约束组合、角色、语言和信息缺失方式，并避免 synthetic paraphrase 造成模式单一。

常见失分点：只做答案改写而 prompt 基本不变。

面试官追问：如何检测 synthetic data 的“模型味”和模板塌缩？

面试题 08 公开真题：CoT 数据怎么构建、筛选与验证？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：Teacher 多样采样 → rule/model verifier → 正确轨迹保留 → 去重与多样性筛选 → 长度/语言/格式约束 → 难度分层 → SFT 或 rejection sampling。

深入理解：final answer 正确不代表 reasoning 合理。数学/代码可用强 verifier；开放任务可组合 answer correctness、过程一致性与对抗检查。还要防止 teacher error 被 student 放大。

常见失分点：把“答案对”当成“链路对”；只依赖一个 judge。

面试官追问：如果两个 CoT 最终都正确，一个短一个长，你怎么选？[I3][I7]

面试题 09 为什么 SFT 经常是 RL 的 cold start？DeepSeek-R1-Zero 又为什么可以跳过？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：RL 需要策略在初始分布中能采到非零 reward 轨迹；若几乎全错，梯度信号稀缺。SFT 可把策略移到可探索区域。

深入理解：但若 base model 足够强、任务可验证且 sampling budget 大，纯 RL 也可能直接激发 reasoning。DeepSeek-R1-Zero 展示了这种可能，同时也出现可读性与语言混用问题，后续 R1 又引入 cold-start data 与多阶段训练。[P8]

常见失分点：把“必须先 SFT”当成铁律。

面试官追问：什么条件决定 pure RL 能否工作？

面试题 10 SFT、DPO 与 Online RL 应该如何选？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：SFT 解决“学会目标行为模板”；DPO 解决“离线偏好排序”；Online RL 解决“通过新采样探索并最大化可计算 reward”。

深入理解：决策顺序：有没有可靠 reward/verifier？是否需要探索新轨迹？数据是静态 pair 还是可以在线生成？系统是否承担 rollout 成本？这比“哪个算法更新”更重要。

常见失分点：用算法热度做选择，而不是由任务约束推出算法。

面试官追问：如果用户偏好主观、但工具任务可验证，你会不会在同一模型上混用 DPO 与 RL？

举一反三 / 章节检查 尝试不看答案，用“任务 → 数据 → reward → exploration → credit → compute → failure → eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第二章 Preference、Reward Model 与 Reward Design

本章目标 完成面试题 11-20。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 11 Reward Model 如何训练？Bradley-Terry 假设是什么？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：对同一 prompt 的 preferred / rejected 输出学习标量 reward，使 $r(\text{chosen}) > r(\text{rejected})$ 。经典损失是 $-\log \sigma(r_w - r_l)$ 。

深入理解：Bradley-Terry 把偏好概率表示为 reward 差的 logistic 函数；它只约束相对顺序，不要求 reward 是可解释的绝对分数。

白板公式 $L_{RM} = -\log \sigma(r_w - r_l)$

常见失分点：把 RM 输出当“真实质量分”；忽略 calibration 与分布外问题。

面试官追问：如果 preference 出现循环 $A > B$ 、 $B > C$ 、 $C > A$ ，标量 RM 会发生什么？

面试题 12 公开真题：PPO 中的 Reward Model 数据从哪里来？

[公开真题] [L2] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：通常由 prompt 上的 policy 多样生成，再由人工或 judge 排序，构造成 preference pairs 训练。

深入理解：RM 数据应覆盖当前 policy 真正会产生的失败模式。policy 演化后，老 RM 可能进入 OOD 区域，出现 reward extrapolation 与 exploitation，因此需要迭代采样/重训或 ensemble。

常见失分点：只说“人工标注”；不说明 policy distribution。

面试官追问：什么时候应刷新 RM？怎样判断 RM 已经失效？[I6]

面试题 13 Preference pair 的 margin、难度和 annotator agreement 为什么重要？

[原理推导] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：winner 与 loser 差距过小会带来高标签噪声；差距过大又可能过于简单。最有价值的是模型容易混淆、但标注相对可靠的 pair。

深入理解：可用 agreement、judge margin、uncertainty、response similarity 和 difficulty 分层抽样，构建既可靠又能提供决策边界信号的数据。

常见失分点：只追求“chosen 很好、rejected 很差”，导致训练信号过易。

面试官追问：如何主动挖掘 hard preference pairs？

面试题 14 如何处理不同标注员之间的 preference disagreement?

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：先区分噪声与真实多样偏好。可采用多标注、软标签、置信度、annotator modeling、分群偏好或保留 Pareto trade-off。

深入理解：若任务本质主观，强行 majority vote 会抹平真实分布；产品应明确是优化平均偏好、特定用户群，还是可个性化策略。

常见失分点：把 disagreement 一律视作脏数据删除。

面试官追问：如何设计一个既保留多样性又可训练的 preference objective?

面试题 15 Reward Model 为什么会被 policy exploit?

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：policy 在做 argmax reward，而 RM 只是有限数据拟合出的 proxy。优化会主动把策略推向 RM 误差最大的尾部。

深入理解：这可类比 Goodhart's Law：proxy 一旦成为优化目标，就会被利用。缓解包括 online data refresh、ensemble uncertainty、rule constraints、held-out judge 与 reward cap。

常见失分点：把 reward hacking 当偶然 bug，而不是优化代理目标的系统性风险。

面试官追问：为什么更强的 optimizer 反而可能让 proxy reward 更不可靠？

面试题 16 公开真题：什么是 Reward Hacking？常见类型有哪些？

[公开真题] [L2] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：模型最大化 proxy reward，却没有提升真实任务目标。常见有 length/verbosity、format、keyword、grader、citation、tool-call、test-case exploit 等。

深入理解：排查时不要只看 train reward，应并列看 held-out task score、human/judge win-rate、length、entropy、reward model disagreement 与异常行为簇。

常见失分点：把 reward 上升当成训练成功。

面试官追问：如何区分 reward hacking 与真正的策略创新？[17]

面试题 17 如何系统检测 Reward Hacking?

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：建立“reward 与真实目标的解耦监控”：proxy reward、独立 verifier、人评/强 judge、分布外测试、长度与风格统计、异常轨迹审计。

深入理解：当 train reward $\uparrow\uparrow$ 、held-out 不变甚至下降、输出长度或某个模式单调上升时，应优先怀疑 proxy exploitation。最有效方法之一是对高 reward 样本做人审/聚类。

常见失分点：只看均值；reward hacking 往往藏在尾部。

面试官追问：怎样设计 dashboard 才能在训练早期发现异常？

面试题 18 Outcome Reward 与 Process Reward 有什么本质差异？

[高频题] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：Outcome Reward 只评价最终结果，保留探索自由但 credit assignment 粗；Process Reward 对中间步骤提供更密集信号，但可能把人工偏见写入搜索路径。

深入理解：在长 CoT 和 Agent 任务中，PRM 能降低信用分配难度；但若过程标签错误，会把错误“正确化”。常见策略是 outcome 为主、selective process 为辅。

常见失分点：认为 PRM 一定优于 ORM。

面试官追问：什么时候宁愿保持 sparse outcome reward？

面试题 19 多个 Reward / Judge 如何组合？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：不要无脑线性加权。先检查量纲、相关性和冲突，再选择 normalization、constraint、gating、lexicographic priority 或 Pareto 方法。

深入理解：例如安全可作为 hard constraint，任务成功为 primary reward，风格与成本做 secondary shaping，比把所有目标放在一个任意加权和更稳。

常见失分点：把不同 reward 原始分数直接相加。

面试官追问：如果 helpfulness 与 safety judge 冲突，如何确定优化优先级？

面试题 20 为什么 rule-based verifier 是 RLVR 的关键基础设施？

[原理推导] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：数学、代码、逻辑等任务可以用规则或可执行环境给低噪声、可扩展、便宜的正确性反馈。

深入理解：GRPO/DAPO 的成功很大程度依赖可规模化 verifier，而不是“RL 算法本身万能”。要同时关注 verifier 的覆盖率、误判与可被利用的漏洞。[P2][P8]

常见失分点：只讨论算法，不讨论 reward 可验证性。

面试官追问：哪些开放任务可以被重新表述成可验证子任务？

第三章 PPO / RLHF：面试必须能手推的经典主干

本章目标 完成面试题 21-30。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 21 公开真题：PPO 是 on-policy 还是 off-policy？为什么答案不能只说一个词？

[公开真题] [L2] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：PPO 通常被视作 on-policy / near-on-policy：数据由 old policy 采样，再对当前 policy 做有限次数更新。

深入理解：importance ratio 修正 current 与 old policy 的差异，但数据复用过多、异步 rollout 或大 batch 会让它越来越偏离 on-policy 假设。

白板公式 $E_{\{\pi_{\theta}\}}[f] = E_{\{\pi_{old}\}}[(\pi_{\theta}/\pi_{old})f]$

常见失分点：说“有 importance sampling 所以 PPO 是 off-policy”。

面试官追问：什么情况下 PPO 的 off-policy bias 会明显增大？[I1]

面试题 22 公开真题：为什么 PPO 需要 Importance Sampling？

[公开真题] [L2] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：数据来自 π_{old} ，但目标在更新 π_{θ} ；用 π_{θ}/π_{old} 将旧分布下的期望校正到新策略附近。

深入理解：它不是魔法：ratio 方差会随策略差异增大，支持集不重合时也无法可靠修正，所以需要 clipping / trust-region 思想限制每次更新幅度。

常见失分点：只背“修正分布偏差”，不解释期望变换。

面试官追问：importance ratio 为什么容易爆？[I1][I11]

面试题 23 PPO clipped surrogate objective 怎么写？min 到底在做什么？

[高频题] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论： $L = E[\min(rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A)]$ 。min 构造 pessimistic surrogate，阻止策略通过过大的概率变化获得不可信的增益。

深入理解：clip 的方向与 A 符号相关；它限制的是“有利方向的过度更新”，不是简单把 ratio 截断后继续优化。

白板公式 $L_{\text{clip}} = E[\min(rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A)]$

常见失分点：把 min 解释成“防止梯度爆炸”。

面试官追问：如果去掉 min 只保留 $\text{clip}(r)A$ ，会有什么差异？[P1]

面试题 24 公开真题：Advantage 为正/负时 clip 分别限制什么？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论： $A > 0$ 时希望提高该 action 概率，但限制 ratio 上界； $A < 0$ 时希望降低概率，但限制 ratio 下界。

深入理解：用数轴或分段函数解释最稳。面试官通常通过这题判断你是真的理解 PPO 目标还是只背公式。

常见失分点：正负方向说反，或只说“限制上下界”。

面试官追问：画出 $A > 0$ 与 $A < 0$ 的 objective 曲线。[I1]

面试题 25 PPO 为什么需要 Critic / Value Model？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：policy gradient 需要 advantage；Critic 估计 $V(s)$ 作为 baseline，降低 Monte-Carlo return 的方差。

深入理解：Critic 是性能与成本的 trade-off：估得准可显著降方差，但 LLM critic 本身几乎是另一个大模型，训练和显存成本高，也是 GRPO 去 critic 的主要动机。

常见失分点：说 critic “负责打 reward”；reward model 与 value model 职责不同。

面试官追问：RM 与 Critic 输入/输出/训练目标分别是什么？

面试题 26 公开真题：GAE 如何计算？ λ 控制什么？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论： $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ ； $GAE = \sum_l (\gamma \lambda)^l \delta_{t+l}$ 。

深入理解： λ 在 bias/variance 之间插值：低 λ 更依赖 bootstrapped value、方差低偏差高；高 λ 更接近 Monte Carlo、偏差低方差高。[P3]

白板公式 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ ； $A_t = \sum (\gamma \lambda)^l \delta_{t+l}$

常见失分点：把 λ 与 γ 混淆。

面试官追问：如果 episode 很长且 value 很差，你会把 λ 调高还是调低？为什么？[I1][I6]

面试题 27 为什么经典 PPO-RLHF 常说需要四个模型？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：Actor 更新策略；Critic 估值；Reward Model 评分；Reference Model 提供 KL anchor。

深入理解：系统成本不只模型权重，还包括 optimizer state、activation、rollout KV cache 与跨卡通信。GRPO 去 critic 后通常能明显降低训练内存，但 rollout 采样仍很贵。

常见失分点：以为 Reference 与 old policy 是同一个概念。

面试官追问： π_{old} 、 π_{ref} 、 $\pi_{rollout}$ 在工程系统里什么时候可能不同？

面试题 28 Reference Model 与 KL penalty 的本质作用是什么？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：常见目标是 $E[R] - \beta \cdot KL(\pi || \pi_{ref})$ ，用 reference 作为语言/行为先验，防止 policy 为追 reward 漂向异常分布。

深入理解：KL 也是稳定性约束与 alignment anchor。注意 RL 算法中的 old policy 用于 trust region/importance ratio，reference 则用于长期行为锚定，两者概念不同。

白板公式 $J(\pi) = E[R] - \beta \cdot KL(\pi || \pi_{ref})$

常见失分点：把所有“旧模型”都叫 ref。

面试官追问：如果不断把 ref 更新成最新 policy，会发生什么？

面试题 29 KL 系数 β 太大或太小会怎样？如何自适应？

[系统设计] [L2]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论： β 太大，policy 几乎不动；太小，容易 policy drift、reward hacking、entropy collapse 与语言质量退化。

深入理解：可采用 target-KL controller：观察实际 KL，高于目标提高 β ，低于目标降低 β 。也可按任务或阶段动态调节。

常见失分点：只靠一次网格搜索固定 β 。

面试官追问：为什么不同 prompt 的合理 KL 可能差异很大？

面试题 30 PPO 在 LLM 后训练中最大的工程问题是什么？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：四模型内存、critic 稳定性、在线 rollout 成本、generation 与 training workload 不匹配，以及 policy freshness。

深入理解：PPO 是“算法 + 分布式系统”的联合问题；当模型到 32B/70B，数据流、权重同步与 KV cache 可能比公式更决定训练效率。

常见失分点：只讨论 hyperparameter。

面试官追问：给你 64 张 GPU，如何划分 rollout 与 learner？

举一反三 / 章节检查 尝试不看答案，用“任务 → 数据 → reward → exploration → credit → compute → failure → eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第四章 DPO 及偏好化：RLHF 目标推出来

本章目标 完成面试题 31-40。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 31 DPO loss 怎么写？四个 log-prob 项各在做什么？

[高频题] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：DPO 比较 chosen 与 rejected 相对 reference 的 log-prob 改变量： $\beta[(\log\pi\theta(y_w)-\log\pi_{\text{ref}}(y_w))-(\log\pi\theta(y_l)-\log\pi_{\text{ref}}(y_l))]$ ，再过 log-sigmoid。

深入理解：它并非简单“提高 chosen、降低 rejected”，而是相对 reference 比较两者隐式 reward 的 margin。[P4]

白板公式 $\Delta = \beta[(\log\pi\theta(y_w)-\log\pi_{\text{ref}}(y_w))-(\log\pi\theta(y_l)-\log\pi_{\text{ref}}(y_l))]$

常见失分点：漏掉 reference 或把序列 logprob 写成单 token。

面试官追问：为什么 DPO 的序列长度偏差值得关注？

面试题 32 DPO 如何从 KL-regularized RLHF 推导出来？

[原理推导] [L4]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：从 $\max_{\pi} E[r] - \beta \text{KL}(\pi || \pi_{\text{ref}})$ 得到最优策略 $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(r/\beta)$ 。重排得 $r = \beta \log(\pi^*/\pi_{\text{ref}}) + C(x)$ ，代回 Bradley-Terry 偏好概率， $C(x)$ 在 pair 差分中消失。

深入理解：这道题的价值在于说明 DPO 不是“拍脑袋的分类损失”，而是把 RLHF 的 reward parameterization 解析化。[P4]

白板公式 $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \cdot \exp(r(x,y)/\beta)$

常见失分点：只背最终 loss，不会讲 partition function 为什么消掉。

面试官追问：DPO 的哪些假设在真实数据里可能不成立？

面试题 33 为什么 DPO 比 PPO 简单很多？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：不需要显式 RM、Critic、在线 rollout、GAE 和 policy optimization loop；训练形态更接近 pairwise classification。

深入理解：简化换来的代价是 offline：无法通过当前 policy 的新探索轨迹持续修正数据分布。

常见失分点：说 DPO “一定比 PPO 好”。

面试官追问：如果已经有高质量 preference log，但算力有限，为什么 DPO 常是合理 baseline？

面试题 34 公开真题：PPO 与 DPO 怎么选？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：如果目标主要是静态偏好和风格对齐，DPO 简洁稳定；如果 reward 可计算、需要探索新策略或环境交互，online RL 更自然。

深入理解：选择要由任务约束推出，而不是算法排名。很多生产 pipeline 会先 SFT/DPO 做行为对齐，再在 verifiable domain 上做 RL。

常见失分点：把“DPO 离线、PPO 在线”说完就停；还要讨论 reward 与 exploration。

面试官追问：开放聊天 + 数学 reasoning 的统一模型如何分阶段？[I6][I10]

面试题 35 DPO 的 offline distribution shift 问题是什么？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：偏好数据来自旧 policy/其他模型，而训练后的 π_θ 会离开该分布；pair 可能不再覆盖模型新出现的错误。

深入理解：因此 DPO 缺少“采样 → 反馈 → 再优化”的闭环。可以用 iterative DPO、online preference collection 或主动数据挖掘改善。

常见失分点：认为训练集越大就自动解决 shift。

面试官追问：如何设计 iterative DPO 数据闭环？

面试题 36 DPO 为什么也可能 overfit 或出现 length bias？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：模型可以不断拉大 chosen/rejected 的 logprob margin，而真实生成质量未必同步提升；若 chosen 系统性更长，还可能学到长度 proxy。

深入理解：应监控 length-controlled win-rate、KL、held-out preference 和 generation metrics，而不只 DPO loss。

常见失分点：只看 pair accuracy。

面试官追问：为什么序列 logprob 天然偏向短序列？如何校正？

面试题 37 DPO 中 β 应如何理解？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论： β 源自 KL-regularized RLHF 的温度/正则尺度，控制 preference pressure 与 reference anchoring 的相对强度。

深入理解： β 不是脱离数据的“学习率替代品”；其有效范围与 pair 难度、模型规模、length normalization 都有关。

常见失分点：把 β 解释成普通 sigmoid 温度即可。

面试官追问： β 改变会怎样影响隐式 reward margin？

面试题 38 KTO 与 DPO 的数据要求有何差异？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：DPO 依赖 chosen/rejected pair；KTO 可从单个样本的 desirable/undesirable 二元信号学习，并使用 prospect-theoretic utility。[P5]

深入理解：当 pair 难采集、只有点赞/点踩式反馈时 KTO 更自然，但仍需考虑标签偏差和正负比例。

常见失分点：说 KTO “不需要偏好数据”；它仍需要人类价值信号，只是不必成对。

面试官追问：什么业务日志天然适合 KTO 形式？

面试题 39 ORPO 为什么可以 reference-free？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：ORPO 将 SFT 与 preference pressure 合并，用 odds ratio 对 favored/disfavored 风格进行对比，不再单独维护 reference model。[P6]

深入理解：它体现了一个方向：不一定要严格分成 SFT 阶段和 preference 阶段；但工程与数据是否适合一体化仍需验证。

常见失分点：只记“省显存”。

面试官追问：什么时候分阶段训练反而更易 debug？

面试题 40 SimPO 相比 DPO 改了什么？为什么使用平均 log probability？

[高频题] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：SimPO 使用 sequence average log probability 作为隐式 reward，reference-free，并加入 target reward margin。[P7]

深入理解：平均 logprob 对长度做归一，更贴近实际生成评分，并降低 reference 计算成本。

常见失分点：把 length normalization 只看作数值技巧；它直接影响偏好目标。

面试官追问：GSPO 的几何平均 ratio 与 SimPO 的平均 logprob 有什么思想共性？

举一反三 / 章节检查 尝试不看答案，用“任务 → 数据 → reward → exploration → credit → compute → failure → eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第五章 GRPO：“去 Critic”到真实工程问题

本章目标 完成面试题 41-50。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 41 公开真题：PPO 与 GRPO 最大区别是什么？

[公开真题] [L2] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：GRPO 用同一 prompt 下多条采样的 group-relative reward 构造 baseline/advantage，从而不训练 critic。

深入理解：这减少了 value model 的显存和训练复杂度，但需要每个 prompt 采 G 条 response，实际是“用 sampling compute 换 critic”。GRPO 最早在 DeepSeekMath 中提出。[P2]

常见失分点：只说“GRPO 没有 value model”，不讲代价。

面试官追问：group size G 对方差和吞吐有什么影响？[I3][I11]

面试题 42 为什么 group-relative baseline 能替代 Critic？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：同一 prompt 的 G 个 Monte-Carlo samples 提供局部相对基线： $A_i = (r_i - \text{mean}(r)) / \text{std}(r)$ 。

深入理解：它不学习跨状态的 value function，而是用“问题竞争”消除 prompt 难度的部分尺度差异。优势是简单；代价是对 group 质量和 reward 方差敏感。[P8]

白板公式 $A_i = (r_i - \text{mean}(r_1..r_G)) / \text{std}(r_1..r_G)$

常见失分点：说它等价于精确 advantage；其实只是 estimator。

面试官追问：如果组内 reward 非常接近，会怎样？

面试题 43 GRPO 去掉 Critic 后，真正的成本转移到了哪里？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：转移到 rollout：每个 prompt 要生成多条长 response；reasoning 场景的 autoregressive decoding 极其昂贵。

深入理解：因此系统优化重点会转向 inference throughput、continuous batching、KV cache、长尾管理、policy versioning 和训练/推理解耦。

常见失分点：认为“少一个模型 = 总成本一定更低”。

面试官追问：什么场景 critic 成本反而值得付？

面试题 44 公开真题：Sequence-level reward 如何传到 token? credit assignment 有什么问题?

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：最简单是把同一 sequence advantage broadcast 给所有 response token，再对每个 token 的 logprob 做 policy gradient。

深入理解：这隐含“整条轨迹 token 共享相同归因”，对长 CoT/Agent 非常粗。错误步骤和关键步骤被同样奖励，推动了 PRM、token weighting 与更细 credit 的研究。

常见失分点：把 broadcast 说成精确归因。

面试官追问：如果答案正确但前半段有大量无效推理，梯度应如何处理？[I1][I8]

面试题 45 为什么 group 内全对或全错时 GRPO 基本没有有效梯度?

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：若 r_i 全相同，组内标准化后 $A_i \approx 0$ ，无法区分哪条轨迹更优。

深入理解：这意味着过易/过难 prompt 虽然消耗 rollout，但不提供 relative learning signal；DAPO Dynamic Sampling 就针对这一问题。[P9]

常见失分点：只认为全错样本无用；全对也可能无 relative signal。

面试官追问：是否应该永远丢掉全对样本？对保持能力有何影响？

面试题 46 Group size G 越大越好吗?

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：G 增大通常能降低 group baseline 方差、提升成功样本概率和多样性，但 rollout 成本、延迟、KV cache 与 queue pressure 线性增加。

深入理解：最优 G 与任务难度和成功率有关。对很难题，G 太小可能全错；对简单题，G 太大浪费。可按 difficulty 自适应采样。

常见失分点：固定 G 不看难度。

面试官追问：怎样用历史 pass rate 设计 adaptive G?

面试题 47 公开真题： π_θ 、 π_{old} 、 $\pi_{rollout}$ 分别是什么？为什么工程中可能不相等?

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论： $\pi_{rollout}$ 是实际生成样本的版本， π_{old} 是优化 ratio 的行为策略快照， π_θ 是当前 learner。同步系统可近似 $\pi_{rollout} = \pi_{old}$ ；异步或大 batch 时会产生版本滞后。

深入理解：一旦不相等，importance ratio、clip 与 on-policy 假设都需要重新审视。

常见失分点：把 reference policy 也混入这三个概念。

面试官追问：如何记录 policy_version 并丢弃过旧 rollout? [I6]

面试题 48 公开真题：batch 很大时为什么 GRPO 会越来越 off-policy?

[公开真题] [L4] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：rollout 生成、reward、排队、batch assemble 需要时间，learner 同期更新多个 step；数据到达 learner 时已来自更旧 policy。

深入理解：缓解方式：缩短 staleness、减少 inner epochs、versioned queue、importance correction、同步 barrier、限制 queue depth、丢弃 stale batch。

常见失分点：只说“batch 大方差小”。

面试官追问：吞吐与 freshness 如何做量化 trade-off? [I6]

面试题 49 如何缓解 policy lag? 同步 RL 与异步 RL 各自适合什么?

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：同步：fresh、目标清晰，但 straggler 严重；异步：吞吐高，但 staleness 与 off-policy bias 增加。

深入理解：工程上常做有界异步：固定最大版本差、队列深度、权重同步频率与 stale sample 策略。

常见失分点：认为“异步一定更快更好”。

面试官追问：如果 rollout 是训练总成本 80%，你会优先优化哪一侧？

面试题 50 为什么 GRPO 特别适合数学/代码等 verifiable tasks?

[原理推导] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：这些任务可以获得稳定的 correctness verifier，避免昂贵且可被 exploit 的主观 RM；group samples 也能提供明确的相对成功信号。

深入理解：真正关键组合是 GRPO + RLVR：算法负责稳定更新，verifier 决定 reward 质量与可扩展性。[P2][P8]

常见失分点：把 GRPO 当成通用聊天对齐的默认算法。

面试官追问：开放问答如何创造“半可验证”reward?

举一反三 / 章节检查 尝试不看答案，用“任务 → 数据 → reward → exploration → credit → compute → failure → eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第六章 DAPO / GSPO : 2026 面试高频前沿

本章目标 完成面试题 51-60。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 51 公开真题：DAPO 相比 GRPO 做了哪些核心改进？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：四项必须脱口而出：Clip-Higher、Dynamic Sampling、Token-level Policy Gradient Loss、Overlong Reward Shaping。[P9]

深入理解：这四项分别对应 entropy collapse、无效组采样、长 CoT 长度 weighting、截断边界 reward noise。高分答案要把 trick 与 failure mode 一一配对。

常见失分点：只背四个名词。

面试官追问：每项如果去掉，训练曲线最可能出现什么症状？[I3][I7]

面试题 52 为什么 DAPO 需要 Clip-Higher？

[原理推导] [L4]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：对称 clip 会较强限制低概率但高 advantage token 的概率上升，可能促使策略多样性快速丢失。DAPO 放宽 upper clip，提升探索并缓解 entropy collapse。[P9]

深入理解：它不是“所有方向都放宽”，而是 decoupled lower/upper clipping。

常见失分点：把 clip-higher 解释为学习率变大。

面试官追问：为什么提高上界不会必然导致 KL 爆炸？还需要哪些监控？

面试题 53 公开真题：Dynamic Sampling 为什么有效？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：过滤组内 accuracy=0 或 1 的 prompt，使每个 batch 保持较稳定数量的有效梯度样本。[P9]

深入理解：它本质上是按“当前策略可学习性”重采样：太易/太难都降低权重，从而把 rollout budget 投到 frontier。

常见失分点：只说“过滤简单题”。全错也会被过滤。

面试官追问：动态采样会不会改变训练分布并造成遗忘？如何补偿？[I7]

面试题 54 长 CoT 下 sequence-level normalization 为什么可能产生长度偏差？

[原理推导] [L4]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：若先对每条 response 做均值再让每条 sequence 权重相等，500 token 与 10k token 的总梯度预算相近，长轨迹单 token 权重被显著稀释。

深入理解：应明确“我们希望等权 sequence，还是等权 token？”这是目标定义问题，不只是实现细节。

常见失分点：认为 length normalization 总是公平。

面试官追问：如何设计一个不偏短也不偏长的 loss aggregation？

面试题 55 Token-level Policy Gradient Loss 解决什么？

[原理推导] [L4]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：DAPO 在 batch 内按 token 聚合，让优化权重与有效 response token 更一致，降低 long-CoT 场景中 per-sequence normalization 带来的偏差。[P9]

深入理解：这会改变不同长度轨迹的相对贡献，应配合 overlong 策略和长度分布监控。

常见失分点：把 token-level 理解成“每个 token 都有独立 reward”。

面试官追问：token aggregation 与 token credit assignment 是一回事吗？

面试题 56 Overlong Reward Shaping 为什么比硬截断惩罚更稳？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：硬阈值会制造 reward cliff：刚超 1 个 token 的正确解可能突然从高 reward 变低 reward，引入与质量无关的高噪声梯度。

深入理解：平滑 shaping 在接近上限时逐渐惩罚，降低边界不连续。[P9]

常见失分点：认为它只是让回答更短。

面试官追问：如果模型学会故意在阈值前提前给答案，会不会产生新 hacking？

面试题 57 公开真题：GSPO 与 GRPO 的核心区别是什么？

[公开真题] [L4] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：GSPO 将 importance ratio、clipping 与优化单位提升到 sequence level，而 GRPO 通常在 token level 处理 ratio。[P10]

深入理解：其 sequence ratio 本质是每 token likelihood ratio 的几何平均，降低长序列与局部数值误差带来的不稳定，并与 sequence reward 更一致。

白板公式 $s_i = (\pi\theta(y_i|x) / \pi_{old}(y_i|x))^{(1/|y_i|)}$

常见失分点：只说“GSPO 更稳定”。

面试官追问：为什么优化单位和 reward 单位的一致性重要？[I3][I11]

面试题 58 GSPO 为什么需要 $1/|y|$ 的长度归一化？

[原理推导] [L4]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：sequence probability 是 token probability 连乘，ratio 的尺度会随长度迅速变化。取 log 后平均、再 exp，相当于几何平均 token ratio，使不同长度 sequence 更可比。[P10]

深入理解：这是目标尺度设计，不只是防 underflow。

白板公式 $\log s_i = (1/|y_i|) \sum_t \log(\pi_t / \pi_{old})$

常见失分点：把长度归一化只理解为数值稳定技巧，而忽略它对不同长度样本权重的影响。

面试官追问：如果不做长度归一，clip 对长序列会有什么行为？

面试题 59 公开真题：为什么 GSPO 对 MoE routing mismatch 更友好？

[公开真题] [L5] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：MoE 的 token-level likelihood 对 expert routing 的微小差异很敏感；GRPO 可能需要 Routing Replay 保持训推一致。GSPO 只依赖 sequence-level likelihood，对局部 token mismatch 更容忍，可减少基础设施负担。[P10]

深入理解：关键不是“MoE 更快”，而是 optimization granularity 对 routing perturbation 的敏感性不同。

常见失分点：忽略推理引擎与训练引擎精度/路由差异。

面试官追问：Routing Replay 为什么有内存与通信成本？[I8]

面试题 60 DAPO 与 GSPO 应如何比较？它们不是同一层面的改进

[系统设计] [L5]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：DAPO 更像围绕 GRPO 的大规模 long-CoT recipe：修 entropy、sampling、length 与 reward noise；GSPO 则直接改变 policy optimization 的 sequence 粒度。

深入理解：可以把 DAPO 看作“工程/目标细化”，GSPO 看作“优化单元重定义”。真实系统也可组合其中思想，而非二选一。

常见失分点：按 benchmark 排名简单选算法。

面试官追问：如果你遇到 MoE 训推 logprob 不一致 + 大量全对组，分别该用哪些思想？

举一反三 / 章节检查 尝试不看答案，用“任务 → 数据 → reward → exploration → credit → compute → failure → eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第七章 Reasoning RL、Verifier 与 Credit Assignment

本章目标 完成面试题 61-70。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 61 为什么 Online RL 可能提升 reasoning，而 SFT 不一定？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：SFT 模仿 teacher 轨迹；online RL 在 reward 指引下从当前 policy 多次采样、探索并提高成功轨迹概率，可以发现训练数据未显式给出的策略组合。

深入理解：RL 的价值包含 elicitation、search 与 policy improvement；但前提是 reward 足够可靠且 exploration 能触达成功区域。[P8]

常见失分点：把 RL 神化成“创造新知识”。

面试官追问：如何证明提升来自 exploration 而不是训练 token 更多？

面试题 62 RL 是“创造能力”还是“激活已有能力”？

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：更准确的答案是两者混合：重权预训练中极低概率行为，同时通过迭代搜索组合出新策略。

深入理解：可以用 behavior support 的观点讨论：RL 很难从完全零支持的行为凭空学习，但能把微弱概率路径放大并组合。

常见失分点：非黑即白回答。

面试官追问：如何用 base model pass@k 与 RL 后 pass@1 判断 elicitation 成分？

面试题 63 为什么数学/代码 RL 比 open-ended chat RL 更容易？

[原理推导] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：前者有客观 verifier，reward 信噪比高；聊天的 helpfulness、truthfulness、style、安全往往主观且多目标冲突。

深入理解：因此“能不能规模化 RL”常由 reward verifiability 决定，而不是 optimizer 是否先进。

常见失分点：只说数学数据更多。

面试官追问：如何把 open-ended 任务拆成可验证子目标？

面试题 64 Process Reward 一定比 Outcome Reward 好吗？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：不一定。PRM 提供更密 credit，但过程标签可能固化人类路径、限制探索并引入额外噪声；ORM 更稀疏却更自由。

深入理解：长 horizon 任务常用 outcome 主目标 + 关键节点 process signal 的折中。

常见失分点：默认步骤越多监督越好。

面试官追问：什么时候 process reward 会伤害 emergent reasoning?

面试题 65 Sparse Reward 如何改善？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：curriculum、难度分层、reward shaping、subgoal verifier、process reward、rejection sampling warm-start、提高 sampling diversity。

深入理解：目标是提高 $P(R>0)$ 并降低 credit variance，但 shaping 不能改变最优策略到错误方向。

常见失分点：随意加 dense reward。

面试官追问：potential-based shaping 为什么更安全？

面试题 66 如何避免模型把“更长 CoT”误学成“更高能力”？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：同时监控 accuracy、length、accuracy/token、pass@k、不同长度 bucket 的收益；采用 efficiency reward、overlong shaping、长度控制 eval 与蒸馏。

深入理解：长 reasoning 有时是探索结果，但长度本身不应成为 proxy。

常见失分点：只设 max_length 粗暴截断。

面试官追问：如何设计 length-controlled A/B evaluation？

面试题 67 Entropy collapse 是什么？为什么 GRPO/DAPO 特别关注？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：策略分布越来越尖，group samples 变得相似，探索消失，relative advantage 信号进一步减少，形成自强化。

深入理解：DAPO Clip-Higher 试图给低概率优质 token 更大的上升空间；还可监控 token entropy、distinctness、pass@k 与 mode coverage。[P9]

常见失分点：只看整体平均 entropy，不看位置/任务分布。

面试官追问：entropy 下降什么时候是健康 convergence？

面试题 68 为什么只看训练 reward 非常危险？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：proxy reward 可能被 exploit，且 reward 分布本身可能漂移。必须联合 held-out verifier、benchmark、human/judge、KL、entropy、length 与 error taxonomy。

深入理解：一条健康训练曲线应展示 reward 与真实任务指标共同改善，而不是 reward 单边上涨。

常见失分点：把 reward 当 ground truth。

面试官追问：什么指标最适合当 early-stop signal？

面试题 69 Rule-based verifier 也会被 hacking 吗？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：会。数学有等价表达/单位/格式漏洞，代码有弱测试、hardcode、超时/环境 exploit。

深入理解：应做 adversarial verifier tests、hidden tests、canonicalization、sandbox 与多 verifier 交叉验证。

常见失分点：认为规则 verifier 天然安全。

面试官追问：如何估计 verifier false positive 对 RL 的危害？

面试题 70 如何设计一个“好 Reward”？

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：四个关键词：目标相关、信号足够、难以作弊、可规模化。必要时再加稳定尺度与可解释拆分。

深入理解：这四者互相冲突：越 dense 越可能泄漏路径，越复杂 judge 越贵且越可被 exploit。reward design 是后训练的核心产品定义。

常见失分点：从公式出发而不是从真实目标出发。

面试官追问：如果 reward 只能满足其中三项，你优先牺牲哪一项？

举一反三 / 章节检查 尝试不看答案，用“任务 → 数据 → reward → exploration → credit → compute → failure → eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第八章 RL 系：Rollout、分布式与显存

本章目标 完成面试题 71-80。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 71 公开真题：一个完整 GRPO 数据流是什么？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：Prompt → rollout 生成 G 个 response → reward/verifier → group normalization → advantages → logprob/importance ratio → policy gradient → actor update → 权重同步 → 下一轮 rollout。

深入理解：回答时把每一步对应到 tensor shape、设备位置与 policy version，会显著区分“读过论文”和“跑过系统”。

常见失分点：漏掉 old policy logprob/weight sync。

面试官追问：哪些数据应该在 rollout 端缓存？[I6]

面试题 72 为什么 RL rollout 比 SFT teacher-forcing 贵？

[原理推导] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：generation 是逐 token autoregressive，无法像 SFT 一样对整条目标序列一次并行前向；长 CoT 让 decode token 成为主要成本。

深入理解：训练侧偏 FLOPs，rollout 侧偏 memory bandwidth + KV cache + sequential latency，两者最佳并行策略不同。

常见失分点：只比较 forward 次数。

面试官追问：为什么 rollout engine 常与 learner 分离？

面试题 73 公开真题：rollout 长尾为什么降低 GPU 利用率？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：同批 response 长度差异大，短序列结束后仍需等待最长序列，造成 decode slot 空闲与同步等待。

深入理解：问题可视为 straggler + dynamic sequence length；在 reasoning RL 中 1k 与 30k token 可同批出现，长尾非常严重。

常见失分点：只说 padding 浪费；decode 阶段更关键是活跃序列数量下降。

面试官追问：如何用 length bucket 和 continuous batching 缓解？[I7][I8]

面试题 74 rollout 长尾有哪些工程解法？

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：continuous batching、dynamic batching、length bucketing、partial rollout、异步调度、token budget、early stop、sequence migration、load balancing。

深入理解：没有单一最佳方案：bucketing 提升同质性但可能延迟；partial rollout 降 straggler 但改变训练数据 freshness。

常见失分点：列技术名词但不讲 trade-off。

面试官追问：如果必须保证严格 on-policy，哪些异步方案不能用？

面试题 75 vLLM 为什么适合 RL rollout?

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：核心是 PagedAttention 管理动态 KV cache、减少碎片与冗余，并结合 continuous batching 提高 decode throughput。[P13]

深入理解：RL 场景尤其需要大量并发、长输出和多样长度，因此 KV cache 管理直接决定可并发的 samples。

常见失分点：只说“vLLM 快”。

面试官追问：PagedAttention 与操作系统虚拟内存的类比是什么？

面试题 76 公开真题：FSDP 与 DDP 的核心区别？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：DDP 每 rank 保持完整参数/优化器，梯度 all-reduce；FSDP 对参数、梯度、optimizer state 做 shard，并在计算前 all-gather、之后 reshard/reduce-scatter。[P12]

深入理解：FSDP 节省内存但增加通信与调度复杂度；FSDP2 以 per-parameter sharding 提升可用性。

常见失分点：说 FSDP 只是“参数切片”。

面试官追问：FULL_SHARD 与 SHARD_GRAD_OP 的通信/显存 trade-off？[I1][I7]

面试题 77 公开真题：ZeRO-1/2/3 分别 shard 什么？

[公开真题] [L2] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：Stage 1 shard optimizer states；Stage 2 再 shard gradients；Stage 3 再 shard parameters。

深入理解：记忆表格只是第一层，第二层要能估算 Adam 场景参数/梯度/一二阶状态的显存占比，并讨论通信成本。[P11]

常见失分点：把 stage 顺序记反。

面试官追问：bf16 参数 + fp32 optimizer 时每参数大约多少字节？[I1]

面试题 78 为什么 PPO/GRPO 系统显存比 SFT 更复杂？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：除了训练模型与 optimizer，还可能并存 reference、critic/RM、rollout KV cache、old logprob、reward batch 与多个 engine 的权重副本。

深入理解：显存规划应拆成 model state、activation、KV cache、temporary all-gather、communication buffer，而不是只看参数量。

常见失分点：只按“参数量 $\times 2$ bytes”估算。

面试官追问：如何定位 OOM 是 activation、KV 还是 FSDP all-gather 峰值？

面试题 79 RL 训推分离如何设计？weight sync 的 trade-off 是什么？

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：rollout workers 优化 decode，learner GPUs 优化训练；learner 更新后需把新权重同步到 rollout。同步越频繁，policy fresh 但通信贵；越稀疏，吞吐高但 staleness 大。

深入理解：大模型可用分片权重广播、增量/异步更新与版本控制。真正指标是 $\text{tokens/sec} \times \text{sample freshness} \times \text{learner utilization}$ 。

常见失分点：只最大化单侧吞吐。

面试官追问：怎么定义“可接受的 policy version lag”？

面试题 80 公开真题：TRL、verl、OpenRLHF 这类框架应该理解到什么程度？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：至少要能画 control flow：rollout、reward、advantage、actor update、distributed strategy、inference integration、weight sync。

深入理解：框架使用经验的含金量来自能解释数据在哪里、谁持有哪份模型、发生 OOM/straggler 时怎么定位，而不是“跑过示例”。

常见失分点：只背 API。

面试官追问：如果让你改成异步 rollout，要修改框架哪几层？ [I6]

举一反三 / 章节检查 尝试不看答案，用“任务 $\rightarrow$ 数据 $\rightarrow$ reward $\rightarrow$ exploration $\rightarrow$ credit $\rightarrow$ compute $\rightarrow$ failure $\rightarrow$ eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第九章 训练稳定性、Eval 与 Debug：项目拷打核心

本章目标 完成面试题 81-90。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 81 公开真题：怎么判断一次 RL 训练“质量达标”？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：分五层：optimization (loss/grad norm)、policy (KL/entropy)、reward (均值/方差/分桶)、capability (pass@1/benchmark)、behavior (judge/human/safety)。

深入理解：训练合格不是 reward 最高，而是 held-out 能力改善、行为没有退化、曲线稳定且结果可复现。

常见失分点：只给最终 benchmark。

面试官追问：你会设置哪些 release gate? [I7]

面试题 82 Reward 一直涨但 benchmark 不涨，怎么排查？

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：优先顺序：reward/verifier 实现 → leakage/hacking → train-eval distribution gap → length/style proxy → benchmark saturation → overfitting。

深入理解：先找 objective 是否对齐，再调 optimizer。抽样审计 top-reward trajectories 往往最快发现问题。

常见失分点：第一反应调 LR。

面试官追问：怎样自动聚类高 reward 但低 task-score 样本？

面试题 83 KL 突然暴涨通常意味着什么？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：可能是 LR/epoch 过大、clip 失效、reward scale 异常、stale rollout、reference/old policy 混淆、logprob 数值错误。

深入理解：应同时看 ratio 分布、clip fraction、policy version、grad norm 和 reward scale，按数据流定位。

常见失分点：只调大 β 。

面试官追问：clip fraction 接近 100% 说明什么？

面试题 84 Entropy 一路下降怎么办？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：先区分健康收敛与 collapse：若 pass@1 上升且 pass@k/diversity 仍好，可能正常；若 reward plateau、group similarity 上升，则要增强 exploration。

深入理解：可调 sampling temperature、clip-higher、entropy bonus、harder prompts、diversity sampling，并检查 reward 是否偏好模板化。

常见失分点：“entropy 越高越好”也错误。

面试官追问：应该监控 token entropy 还是 sequence diversity？

面试题 85 Reward variance 很大怎么办？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：先按 task/difficulty/length 分桶，判断是异质性还是 judge 噪声。再考虑 per-task normalization、clipping、calibration、group baseline 或 reward decomposition。

深入理解：简单全局标准化可能掩盖不同任务尺度问题。

常见失分点：只做 z-score。

面试官追问：如果不同任务 reward 量纲不同，如何做 multi-task RL？

面试题 86 为什么离线 benchmark 不能完全代表线上？

[高频题] [L2]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：用户分布包含多轮、长上下文、工具失败、歧义、混合语言与对抗输入，往往不同于静态 benchmark。

深入理解：评测体系应是 offline benchmark + human/judge + shadow/online A/B + failure-set regression。

常见失分点：把 leaderboard 作为产品指标。

面试官追问：怎样防止在线 A/B 被 latency/长度等混杂因素影响？

面试题 87 怎么做后训练 ablation 才可信？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：逐项增加变量，控制 rollout tokens、training steps/compute、seed、eval protocol 与初始 checkpoint。

深入理解：例如 DAPO 应 baseline GRPO → +Clip-Higher → +Dynamic Sampling → +Token PG → +Overlong，而不是 full recipe 对 baseline 一把梭。

常见失分点：只控制 step，不控制 generated tokens。

面试官追问：为什么 RL ablation 特别需要报告 variance？

面试题 88 为什么 SFT 变好但 RL 可能变差？

[原理推导] [L4]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：SFT 可能降低 entropy、变得过度 deterministic，导致 RL exploration space 变窄；也可能过拟合一种 reasoning style。

深入理解：所以最佳 SFT checkpoint 与最佳 RL init 的目标不同。应通过 base pass@k、entropy 与 exploration eval 选择 RL 起点。

常见失分点：认为先把 SFT 做到最强一定更好。

面试官追问：如何定义 “RL-friendly checkpoint” ？

面试题 89 如何构建 hard-example data flywheel?

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：Model → Eval → Failure mining → 聚类/归因 → 生成/标注/验证 → SFT/Preference/RL → Regression → 新模型。

深入理解：关键是根据当前 frontier 动态增加边际训练信号，而不是随机扩数据。要记录 failure taxonomy 与每轮修复率。

常见失分点：只做“错题加入训练集”，导致 contamination 和过拟合。

面试官追问：如何划分永久 held-out failure set?

面试题 90 如何判断问题来自数据、算法还是系统实现?

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：训练集也学不上：先查实现/优化；train 好 eval 差：查数据泛化；reward ↑ task ↓：查 objective；单卡正确多卡异常：查 distributed；同步好异步坏：查 staleness。

深入理解：建立最小可复现实验和 invariants，比盲调超参有效。

常见失分点：一次同时改多个变量。

面试官追问：请设计一个 GRPO loss 数值单测。

举一反三 / 章节检查 尝试不看答案，用 “任务 → 数据 → reward → exploration → credit → compute → failure → eval” 重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

第十章 Agentic RL 与项目系统设计：三面真正拉开差距

本章目标 完成面试题 91-100。要求不是记忆名词，而是把每个方法与其输入、目标函数、failure mode 和工程代价连接起来。

面试题 91 公开真题：什么是 Agentic RL？与单轮 reasoning RL 的状态空间有何不同？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：Agent trajectory 是 state $\rightarrow$ action/tool $\rightarrow$ observation $\rightarrow$ next action 的环境交互，不只是一次 $x \rightarrow y$ 。

深入理解：状态包含对话、工具输出、环境状态和历史动作；horizon 更长、reward 更迟、执行可能有副作用，因此 credit、成本与安全都更复杂。

常见失分点：把 Agentic RL 说成“对 function calling 做 GRPO”。

面试官追问：multi-turn rollout 的终止条件如何定义？[I1][I7]

面试题 92 公开真题：Agentic RL 的 credit assignment 怎么做？

[公开真题] [L4] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：最粗是 terminal reward broadcast；更细可用 step reward、tool validity、process verifier、value/advantage decomposition、hierarchical credit。

深入理解：关键是避免把最终成功错误归因给所有 token。对工具任务可把 action span 作为更自然的 credit unit。

常见失分点：把每一步都手工打 dense reward。

面试官追问：action-level advantage 如何回传到 tool-call token？[I7][I8]

面试题 93 公开真题：Tool Calling / Function Calling 数据怎么构造？

[公开真题] [L3] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：覆盖“是否需要工具 $\rightarrow$ 选哪个工具 $\rightarrow$ 参数 $\rightarrow$ 观察 $\rightarrow$ 继续推理 $\rightarrow$ final”，还必须有不该调用、参数错误、工具失败、结果冲突等 negative cases。

深入理解：只训练正确 tool call 会造成 over-calling；Agent 数据的难点是状态转移与异常恢复，不只是 JSON 格式。

常见失分点：只做 schema 格式 SFT。

面试官追问：如何构造工具失败后的 recovery trajectories？

面试题 94 Agent reward 应如何设计？

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：任务成功做 primary；invalid action/safety 可做 hard penalty/constraint；调用成本、步骤数、延迟做 secondary objective。

深入理解：避免任意线性加权把主目标稀释。对于真实环境，reward 还要考虑可重复性和执行副作用。

常见失分点：每调用一次工具都给正奖励。

面试官追问：成功率与成本冲突时如何做 Pareto eval？

面试题 95 如何防止 Agent 为了 reward 无限调用工具或重复搜索？

[系统设计] [L3]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：设置 task-success 主奖励、per-call cost、step budget、duplicate-action penalty、loop detection、invalid-call 约束。

深入理解：但不能只靠惩罚，否则模型会学“不调用”。应确保不调用时无法获得任务 success reward。

常见失分点：把 max_steps 当唯一方案。

面试官追问：如何检测循环是必要重试还是无效重复？

面试题 96 Multi-turn RL 与 single-turn RL 最大区别是什么？

[原理推导] [L3]

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：multi-turn horizon 长、environment non-stationary、trajectory variable-length、credit 更稀疏，且 rollout 与工具延迟显著增加。

深入理解：系统还要保存/恢复环境 state，并处理 tool timeout、partial observation 与并行 episode scheduling。

常见失分点：只说上下文更长。

面试官追问：多轮训练如何避免 context 越滚越大？

面试题 97 公开真题：长程任务为什么可能选 GRPO 而不是 PPO？

[公开真题] [L4] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：长 horizon 的 value model 难训练且成本高；若有可靠 terminal verifier，用 group-relative outcome 可能更简单。

深入理解：但这不是绝对结论：若有高质量 dense value/process signal，critic 仍可能明显降低 variance。算法选择取决于 reward 与 horizon，而不是流行度。

常见失分点：回答“GRPO 一定更适合长程”。

面试官追问：哪些长程任务反而更需要 critic？[I12]

面试题 98 如何给 Agent 过程打 reward，而不把探索路径写死？

[系统设计] [L4]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：只对可验证关键事件给予 selective shaping：合法工具选择、关键证据获得、状态推进；最终 task success 保持主导。

深入理解：过程 reward 应奖励“进展”而非固定步骤顺序，避免把人工脚本当唯一最优路径。

常见失分点：逐步照标准答案打分。

面试官追问：如何定义 progress metric 而非 path imitation？

面试题 99 系统设计题：如果让你做一个 70B reasoning model 的完整 Post-Training pipeline？

[系统设计] [L5]

面试官为什么问：考察你是否能在信息不完备时做工程权衡：先定义约束和成功指标，再选择算法与系统。

核心结论：Base → 高质量 instruction SFT → reasoning cold-start → RLVR (math/code/logic) → general preference → safety → Agentic RL → failure mining → iterative post-training。

深入理解：每一阶段都要有独立目标、数据、reward、release gate 与 regression set。不要把所有目标一次混进 RL，否则很难定位冲突。

常见失分点：只列算法名，不讲阶段依赖和 eval。

面试官追问：每阶段哪些 checkpoint 应冻结/复用？如何避免 alignment tax？

面试题 100 终极项目题：为什么你的项目选 GRPO/DAPO/GSPO，而不是 PPO/DPO？

[公开真题] [L5] · 来源见题尾索引标记与附录

面试官为什么问：考察你是否能把概念放进完整后训练链路，并从目标函数、数据分布与系统约束解释选择，而不是复述论文摘要。

核心结论：按“任务 → 数据 → reward → exploration → credit → compute → staleness”推导选择。例如数学 reasoning 有可靠 verifier、需要在线探索、32B critic 昂贵，所以 GRPO 合理；若出现全对/全错组、熵塌缩、长 CoT weighting，再引入 DAPO；若 MoE routing/训推 logprob mismatch 与长期稳定性突出，再考虑 GSPO。

深入理解：这道题不是考算法记忆，而是考研究判断：每个算法必须对应一个具体约束或 failure mode。

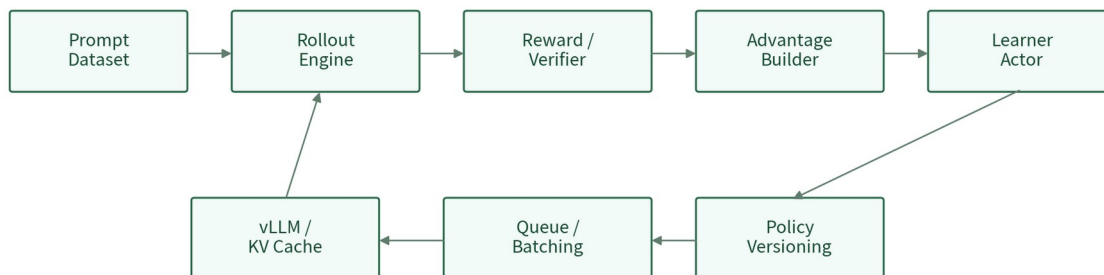
常见失分点：“因为论文效果更好/更新”。

面试官追问：请给出你项目的三项关键 ablation，证明算法选择是必要的。[I3][I7][I11]

举一反三 / 章节检查 尝试不看答案，用“任务 → 数据 → reward → exploration → credit → compute → failure → eval”重新讲本章。若能把 3 个以上问题串成一条因果链，而不是逐题背诵，本章才算掌握。

: RL 系统数据流与瓶颈定位

大模型 RL 系统：真正的瓶颈往往在 rollout 与 policy freshness



系统设计四问：① 谁生成？② 谁打分？③ 数据多久才被 learner 使用？④ 更新后的权重多久同步到 rollout？

长尾、KV cache、continuous batching、异步队列、staleness、训推 logprob 一致性，都是后训练面试的高频工程追问。

系统排障顺序 先确认 policy version 与数据流正确，再看 reward/verifier，再看 ratio/clip/advantage 数值，最后才调学习率与 batch。大量“算法不收敛”其实是系统 staleness 或数据 bug。

高频 20 题：有限时间优先打穿

- 01. Q02 SFT 到底在学什么？为什么说它是 behavior cloning？
- 02. Q03 公开真题：SFT 数据如何筛选和采样？
- 03. Q10 SFT、DPO 与 Online RL 应该如何选？
- 04. Q11 Reward Model 如何训练？Bradley-Terry 假设是什么？
- 05. Q16 公开真题：什么是 Reward Hacking？常见类型有哪些？
- 06. Q21 公开真题：PPO 是 on-policy 还是 off-policy？为什么答案不能只说一个词？
- 07. Q23 PPO clipped surrogate objective 怎么写？min 到底在做什么？
- 08. Q24 公开真题：Advantage 为正/负时 clip 分别限制什么？
- 09. Q26 公开真题：GAE 如何计算？ λ 控制什么？
- 10. Q27 为什么经典 PPO-RLHF 常说需要四个模型？
- 11. Q31 DPO loss 怎么写？四个 log-prob 项各在做什么？
- 12. Q32 DPO 如何从 KL-regularized RLHF 推导出来？
- 13. Q34 公开真题：PPO 与 DPO 怎么选？
- 14. Q41 公开真题：PPO 与 GRPO 最大区别是什么？
- 15. Q42 为什么 group-relative baseline 能替代 Critic？
- 16. Q44 公开真题：Sequence-level reward 如何传到 token？credit assignment 有什么问题？
- 17. Q51 公开真题：DAPO 相比 GRPO 做了哪些核心改进？
- 18. Q57 公开真题：GSPO 与 GRPO 的核心区别是什么？
- 19. Q71 公开真题：一个完整 GRPO 数据流是什么？
- 20. Q100 终极项目题：为什么你的项目选 GRPO/DAPO/GSPO，而不是 PPO/DPO？

训练目标 这 20 题要达到“可讲 5-10 分钟”：定义 → 公式 → why → failure → 工程 → 追问。其余 80 题至少做到 60-90 秒结构化回答。

五条模拟面试路线：从一面到三面

路线 A · RL 基础白板

题链：21 → 22 → 23 → 24 → 26 → 25 → 28 → 29

训练目标：目标：PPO 公式不能断链。

路线 B · Reasoning RL

题链：41 → 42 → 44 → 45 → 51 → 52 → 53 → 55 → 57

训练目标：目标：能从 GRPO failure 推到 DAPO/GSPO。

路线 C · 工程系统

题链：71 → 72 → 73 → 74 → 75 → 79 → 48 → 49

训练目标：目标：能讨论吞吐、staleness 与权重同步。

路线 D · Reward & Eval

题链：11 → 15 → 16 → 17 → 18 → 20 → 68 → 82

训练目标：目标：不被“reward 上升”迷惑。

路线 E · 项目终面

题链：3 → 8 → 10 → 50 → 60 → 81 → 90 → 99 → 100

训练目标：目标：用项目约束自然推出算法选择。

项目题万能答法：不是 STAR，而是“研究决策链”

1. **Problem**：真实业务/研究问题是什么？为什么现有 baseline 不够？
2. **Metric**：成功标准是什么？离线、在线、成本、安全分别怎么量化？
3. **Data**：数据从哪里来？如何筛选？覆盖哪些 failure modes？
4. **Reward**：reward/verifier 为什么可信？怎样防 hacking？
5. **Algorithm**：为什么选 SFT / DPO / PPO / GRPO / DAPO / GSPO？替代方案为什么不选？
6. **System**：模型多大、多少卡、rollout/learner 怎么分、吞吐瓶颈是什么？
7. **Failure**：训练中真正遇到过什么坏现象？你如何定位？
8. **Ablation**：如何证明提升来自你设计的设计，而不是更多 compute/data？
9. **Lessons**：如果重做一次，你会在哪个环节重构？

终面识别度 能讲出“一个失败案例 + 一次错误假设 + 如何用实验推翻 + 最终修正”，通常比罗列十个技术名词更有说服力。

附录 A：资料来源与“真题”口径

口径说明：论文/官方技术报告用于算法事实与公式；公开面经用于证明某类问题确实在面试中出现。面经是候选人自述，不代表企业官方题库，也不能保证题目逐字一致。

- [P1] Schulman et al., Proximal Policy Optimization Algorithms, arXiv:1707.06347 [链接](#)
- [P2] Shao et al., DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024 [链接](#)
- [P3] Schulman et al., High-Dimensional Continuous Control Using Generalized Advantage Estimation, 2015 [链接](#)
- [P4] Rafailov et al., Direct Preference Optimization: Your Language Model is Secretly a Reward Model, 2023 [链接](#)
- [P5] Ethayarajh et al., KTO: Model Alignment as Prospect Theoretic Optimization, 2024 [链接](#)
- [P6] Hong et al., ORPO: Monolithic Preference Optimization without Reference Model, 2024 [链接](#)
- [P7] Meng et al., SimPO: Simple Preference Optimization with a Reference-Free Reward, 2024 [链接](#)
- [P8] Guo et al., DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning, Nature 2025 [链接](#)
- [P9] Yu et al., DAPO: An Open-Source LLM Reinforcement Learning System at Scale, 2025 [链接](#)
- [P10] Qwen Team, GSPO: Towards Scalable Reinforcement Learning for Language Models, 2025 [链接](#)
- [P11] Rajbhandari et al., ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, 2019 [链接](#)
- [P12] PyTorch Documentation, FullyShardedDataParallel / FSDP2, accessed 2026 [链接](#)
- [P13] Kwon et al., Efficient Memory Management for Large Language Model Serving with PagedAttention, 2023 [链接](#)
- [P14] Ouyang et al., Training language models to follow instructions with human feedback, 2022 [链接](#)
- [I1] 牛客：百度 大模型后训练 一面（2026-03-18）：PPO/GAE/GRPO/FSDP/ZeRO/Agentic RL [链接](#)
- [I2] 牛客：百度 大模型后训练实习生 二面（2026-03-20）：GRPO 数据流、policy version、TRL/verl [链接](#)
- [I3] 牛客：字节 大模型算法岗（2026-04/08）：GRPO/DAPO/GSPO、CoT 数据 [链接](#)
- [I4] 牛客：字节 27 暑期：Agentic RL、token credit、rollout 长尾、MoE routing、GSPO [链接](#)
- [I5] 牛客：百川医疗大模型后训练：PPO/GRPO、长程任务算法选择 [链接](#)
- [I6] 牛客：百川大模型算法二面：DPO、PPO RM/GAE、GRPO [链接](#)
- [I7] 牛客：阿里国际 AI 应用算法：SFT 筛选、GRPO、reward design/hacking、训练质量 [链接](#)
- [I8] 牛客：字节大模型算法岗面经：rollout 长尾、MoE routing mismatch、GSPO [链接](#)
- [I9] 牛客：字节 26 校招广告算法：SFT、GRPO reward/hacking、CoT [链接](#)
- [I10] 牛客：阿里大模型算法：SFT/Post-Training、DPO/PPO/GRPO、Agent [链接](#)
- [I11] 牛客：字节大模型算法一面（2026-08）：PPO vs GRPO、DAPO/GSPO、重要性采样 [链接](#)
- [I12] 牛客：百川医疗大模型后训练（2026-08）：长程任务为什么 GRPO [链接](#)

附录 B：公开真题索引

题号	题目	公开面经来源
Q03	公开真题：SFT 数据如何筛选和采样？	I7
Q08	公开真题：CoT 数据怎么构建、筛选与验证？	I3/I9
Q12	公开真题：PPO 中的 Reward Model 数据从哪里来？	I6
Q16	公开真题：什么是 Reward Hacking？常见类型有哪些？	I7/I9
Q21	公开真题：PPO 是 on-policy 还是 off-policy？为什么答案不能只说一个词？	I1
Q22	公开真题：为什么 PPO 需要 Importance Sampling？	I1/I11
Q24	公开真题：Advantage 为正/负时 clip 分别限制什么？	I1
Q26	公开真题：GAE 如何计算？ λ 控制什么？	I1/I6
Q34	公开真题：PPO 与 DPO 怎么选？	I6/I10
Q41	公开真题：PPO 与 GRPO 最大区别是什么？	I3/I11/I12
Q44	公开真题：Sequence-level reward 如何传到 token？credit assignment 有什么问题？	I1/I4/I8
Q47	公开真题： $\pi\theta$ 、 π_{old} 、 $\pi_{rollout}$ 分别是什么？为什么工程中可能不相等？	I2
Q48	公开真题：batch 很大时为什么 GRPO 会越来越 off-policy？	I2
Q51	公开真题：DAPO 相比 GRPO 做了哪些核心改进？	I3/I8/I11
Q53	公开真题：Dynamic Sampling 为什么有效？	I8
Q57	公开真题：GSPO 与 GRPO 的核心区别是什么？	I3/I11
Q59	公开真题：为什么 GSPO 对 MoE routing mismatch 更友好？	I4/I8
Q71	公开真题：一个完整 GRPO 数据流是什么？	I2
Q73	公开真题：rollout 长尾为什么降低 GPU 利用率？	I4/I8
Q76	公开真题：FSDP 与 DDP 的核心区别？	I1/I8
Q77	公开真题：ZeRO-1/2/3 分别 shard 什么？	I1
Q80	公开真题：TRL、verl、OpenRLHF 这类框架应该理解到什么程度？	I2
Q81	公开真题：怎么判断一次 RL 训练“质量达标”？	I7
Q91	公开真题：什么是 Agentic RL？与单轮	I1/I8

	reasoning RL 的状态空间有何不同？	
Q92	公开真题：Agentic RL 的 credit assignment 怎么做？	I4/I8
Q93	公开真题：Tool Calling / Function Calling 数据怎么构造？	I10
Q97	公开真题：长程任务为什么可能选 GRPO 而不是 PPO？	I5/I12
Q100	终极项目题：为什么你的项目选 GRPO/DAPO/GSPO，而不是 PPO/DPO？	I3/I7/I11

如何使用真题索引 不要追求“押中逐字原题”。真实面试会沿着项目继续追：为什么、公式、反例、failure、系统。真题的价值是校准考察深度，而不是把准备变成背题。

附录 C : 14 天冲刺计划

日期	范围	产出
Day 1	Q1-10	SFT / 数据; 完成自己的数据 pipeline 图
Day 2	Q11-20	RM / Reward; 整理一个 reward hacking 案例
Day 3	Q21-26	PPO/GAE 白板
Day 4	Q27-34	KL + DPO 推导
Day 5	Q35-43	Preference family + GRPO
Day 6	Q44-50	credit / off-policy / RLVR
Day 7	Q51-60	DAPO/GSPO, 手画演化图
Day 8	Q61-70	Reasoning / verifier / entropy
Day 9	Q71-80	系统、FSDP、vLLM、rollout
Day 10	Q81-90	Eval / debug; 做 3 个故障树
Day 11	Q91-100	Agentic RL / 70B pipeline
Day 12	20 高频题	每题 5 分钟录音复盘
Day 13	五条模拟路线	连续追问, 不看答案
Day 14	项目答辩	Q100 + 个人项目 30 分钟 mock

最后标准 面试前不要再加新算法名。最后 48 小时只做三件事: 手写核心公式、讲项目 failure、模拟连续追问。